



Estimation of Circular Distributions with Probabilistic Programming

Diogo Almeida

Faculty of Sciences of the University of Lisbon

João Pedro Neto 

LASIGE, DI, Faculdade de Ciências, Universidade de Lisboa

Abstract

This work is motivated by the gap between circular statistical theory and its practical application in the context of probabilistic programming. The project focuses on the specification and estimation of circular models using Bayesian inference, and on building a modular and intuitive Python library. The library will provide algorithms to describe various models based on circular distributions, and will use an existing probabilistic programming tool from the literature, Stan. This library is a public service, organized, tested, validated, and well-documented, making these models accessible to a wider audience.

The document will describe a Python package, `stan_circular_inference`, that provide the whole Bayesian inference pipeline and implements distributions such as the Cardiod and the Wrapped Cauchy.

Keywords: circular data, circular distributions, probabilistic programming, Bayesian inference, Python.

1. Introduction

1.1. Motivation

Circular Statistics is a branch of statistics dealing with cyclic data which is usually represented as points on the circumference of the unit circle. Despite being much less used than linear data, circular data are, nonetheless, all around us, in angles, in data representing cyclic time, like processes that repeat daily, seasonally, or on a yearly basis. The fact that the

range of this type of data has a circular support, makes it difficult to use standard linear methods. If, say, some event occurred repeatedly at times 0:04, 23:56, 0:01, and 23:58, only seen in the perspective of a circumference does it make sense to state that these are all close observations. Computing the mean of the data in a linear setting, for example, would provide a totally unreasonable estimation near 12:00.

The growing volume of available data demands increasingly robust methods to transform it into knowledge. This is true for both linear data and circular data. However, the number of tools concerning the latter is very few compared with the former. More statistical tools are needed to deal with circular methods.

Another increasingly important concern is interpretability: to understand what is the meaning of the results provided by the inferential process. In this context, statistics provides a better methodological context than machine learning (ML) by forcing the scientist to design a probabilistic/statistical model that reflects the structure of the scientific model underlining the given problem. In contrast, ML usually provides out-of-the-box general frameworks for learning patterns that excel at prediction, but fail at understanding causal processes.

In the context of this article, the focus was on the design of probabilistic generative models following the best practices and precepts of Bayesian statistics. As we will see, these models are best seen as templates that the user will realize after providing the necessary arguments, in order to adapt them for the inferential process of their datasets.

1.2. Purpose

stan_circular_inference is a Python module that implements Bayesian models, performing inference through well-established probabilistic programming tools via the **pystan** module. It provides support from model specification to visualisation, thereby reducing the user's need to write general-purpose programming language (GPL) code. Instead, the user can focus on the domain-specific language (DSL) **Stan** and on interpreting the results.

1.3. Usage

As a use case, let's consider a dataset containing wind direction and wind speed measurements. The response variable is the wind direction, measured in degrees, and the covariate is the wind speed. Wind direction is a circular variable modeled here by the von Mises distribution, the circular analogue of the normal distribution.

The following Python code loads the data, preprocesses it, and fits a Bayesian model using the **stan_circular_inference** package, which provides a service for interfacing with **Stan**. First, we import the necessary libraries and read the dataset.

```
import numpy as np
import pandas as pd
from stan_circular_inference.service.bayesian_inference import \
    BayesianInferenceService

df = pd.read_csv('datasets/wind_dataset.csv')
df = df[(df['wind_dir'] != -9999) | (df['wind_speed'] != -9999)].dropna().copy()
```

The dataset originally contains missing value codes -9999 for both variables. Rows with

missing values are removed. Since wind direction is circular, we convert the degrees to radians and map them to the interval $[0, 2\pi)$.

```
df['wind_dir'] = np.radians(df['wind_dir']) % (2*np.pi)
```

As stated, we model the wind direction Y_i (in radians) conditional on wind speed v_i using a Von Mises distribution. The mean direction is allowed to depend linearly on wind speed:

$$Y_i \sim \text{VonMises}(\mu_i, \kappa), \quad \mu_i = \mu + \beta v_i,$$

where μ is the baseline mean direction, β quantifies the effect of wind speed on the mean direction, and κ is the concentration parameter (analogous to the inverse of variance). The following priors are adopted:

$$\mu \sim \text{Uniform}(0, 2\pi), \quad \kappa \sim \text{Exponential}(0.1), \quad \beta \sim \text{Uniform}(0, 1).$$

The Stan program is written as a multi-line string.

```
model = """
  data {
    int<lower=0> N;
    vector[N] direction;
    vector[N] velocity;
  }

  parameters {
    real<lower=0, upper=2*pi()> mu;
    real<lower=0> rho;
    real<lower=0, upper=1> beta;
  }

  model {
    mu ~ uniform(0, 2*pi());
    rho ~ uniform(0.25, 0.25);
    beta ~ uniform(0, 1);

    for(n in 1:N){
      real mu_final = mu + beta * velocity[n];
      direction[n] ~ cardioid(mu_final, rho);
    }
  }
  """
```

The Stan DSL does not feature a built-in implementation of the Cardioid and Wrapped Cauchy distributions. Therefore, within the initialisation method (`init`) of the `BayesianInferenceService`

class, a preprocessing step is performed on the model to add the implementation of these distributions should the user employ them; this process is entirely transparent to the user. Consequently, the keywords `'cardioid'`, `'cardioid_lpdf'`, `'wrapped_cauchy'`, and `'wrapped_cauchy_lpdf'` become reserved words within this library.

The upper bound of μ is set to 2π . The prior for β is restricted to the unit interval, reflecting the assumption that wind speed has a non-negative and limited influence on the mean direction.

An instance of type `BayesianInferenceService` is created with the model string,

```
service = BayesianInferenceService(model)
```

The data are prepared as a dictionary containing the number of observations, the direction vector (in radians), and the wind speed vector.

```
model_data = {
    'N': len(df['wind_dir'].values),
    'direction': df['wind_dir'].values,
    'velocity': df['wind_speed'].values
}
```

The posterior samples are obtained using the `get_pystan_statistics` method, which runs Stan's sampler (e.g., NUTS) and returns summary statistics for the parameters of interest. The argument `sample_amount=5000` determines the number of posterior draws (after warm-up).

```
values = service.get_pystan_statistics(
    data=model_data,
    parameters=['mu', 'kappa', 'beta'],
    sample_amount=5000
)
```

The returned object `values` contains point estimates (e.g., posterior mean) and posterior intervals. To visualise the posterior distributions, the service provides a `multiple_graphics` method. The argument `parameters_type` indicates which parameters are circular (here only μ) so that appropriate plots (rose diagrams) are generated.

```
service.multiple_graphics(
    values,
    min_val=0,
    max_val=2*np.pi,
    parameters_type=[True, False, False]
)
```

The resulting plots in Figure 1 help assess convergence and the shape of the posterior distributions for μ , κ and β .

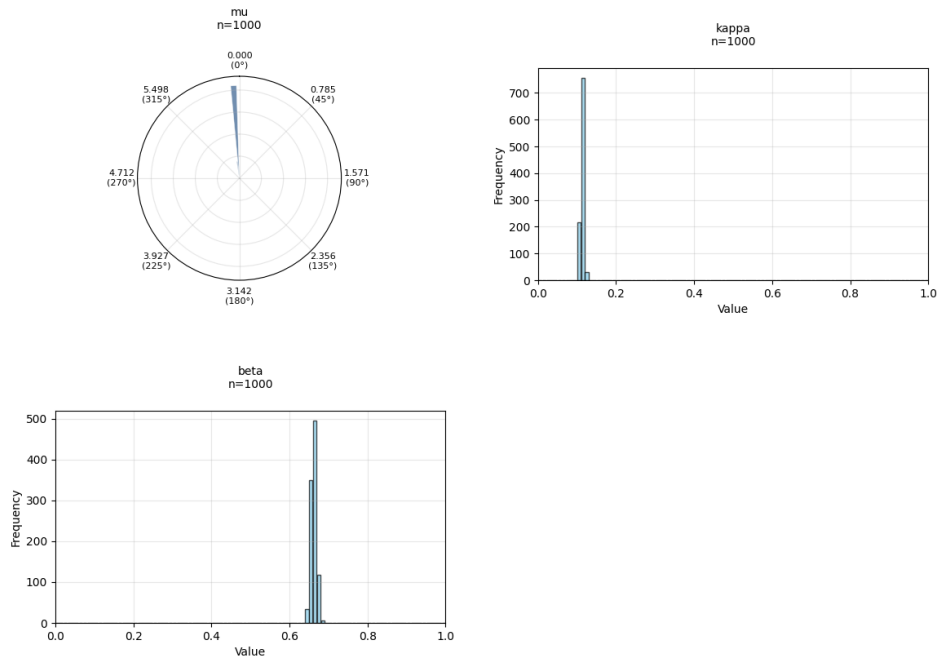


Figure 1: Inferred values of the parameters: μ, κ, β

The model links the mean direction linearly to wind speed. If the posterior of β is concentrated away from zero, it indicates that wind speed systematically shifts the prevailing wind direction. The concentration parameter κ quantifies the dispersion around the mean direction: larger κ corresponds to less dispersion (more reliable direction). Because the response is circular, careful interpretation is required; for example, a change in μ of π radians corresponds to a reversal of direction.

1.4. Other related packages

PyMC [Abril-Pla, Andreani, Carroll, Dong, Fonnesbeck, Kochurov, Kumar, Lao, Luhmann, Martin, Osthege, Vieira, Wiecki, and Zinkov \(2023\)](#) is a probabilistic programming library for Bayesian inference in Python. It supports the specification of complex statistical models through a syntax close to mathematical notation, includes distributions such as the von Mises for circular data, and implements Markov Chain Monte Carlo samplers including the No-U-Turn Sampler (NUTS). The library is employed for custom model construction, posterior inference, and diagnostic evaluation.

PyCircular [Albahnsen \(2019\)](#) addresses general circular data analysis in Python, focusing on preprocessing, visualisation, and machine learning. It provides transformations of angular variables, generation of periodic features (sine and cosine), and circular kernels. Although it offers basic distribution functions (e.g., von Mises), its primary orientation is machine learning integration.

pycircstat2 [circstat \(2024\)](#) implements classical (frequentist) circular statistics. A modern successor to the original **pycircstat**, it supplies functions for descriptive statistics (circular mean, concentration), hypothesis tests (Rayleigh, Kuiper), and circular visualisations (rose diagrams, circular histograms). The library is confined to standard frequentist analysis and

excludes Bayesian methods.

Oriana Kovach Computing Services (2025) is a commercial Windows application with a graphical user interface dedicated to circular and directional data analysis. It enables descriptive statistics, hypothesis tests (uniformity, comparison of circular means), and circular regression via point-and-click operations. The software handles angles, times, and dates, supports second-order analyses, and rests on frequentist principles.

PAST Hammer, Harper, and Ryan (2020) (PAleontological STatistics) is a free, cross-platform package for scientific data analysis, prevalent in palaeontology, biology, and geosciences. Its circular statistics module offers descriptive measures, hypothesis tests (e.g., Rayleigh’s test), and rose diagrams. Beyond circular data, PAST provides multivariate analysis, time series, and curve fitting. The package follows classical frequentist methodology.

2. Probabilistic Models

We consider herein three well-known circular distributions, namely the Von Mises, the Cardiod and the Wrapped Cauchy. All these are characterized by two parameters: a location parameter $\mu \in [0, 2\pi[$ and a non-negative parameter that controls the concentration around μ . All have a circular uniform distribution as a special case, and all are symmetric about their unique mode μ . This notion of symmetry on the circle is referred to as reflective symmetry.

From a Bayesian perspective, the goal is to infer the posterior distribution of the unknown parameters given the observed data. Bayes’ theorem provides the foundation for this inference:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta) p(\theta)}{p(\mathcal{D})}$$

where:

- $p(\mathcal{D} | \theta)$ - the *likelihood*, which quantifies how well the model with parameters θ explains the observed data $\mathcal{D}$;
- $p(\theta)$: the *prior* - which represents the knowledge or prior beliefs about the parameters before observing the data;
- $p(\theta | \mathcal{D})$ - the posterior, which expresses the updated knowledge about the parameters after incorporating the evidence provided by the data;
- $p(\mathcal{D})$ - the evidence, a normalization constant that ensures the posterior is a valid probability distribution.

Since $p(\mathcal{D})$ is constant with respect to θ , we can simplify:

$$p(\theta | \mathcal{D}) \propto p(\mathcal{D} | \theta) p(\theta)$$

The posterior distribution represents our updated knowledge about the parameters after observing the data.

Each of the implemented distributions will be addressed individually in this section in order to explain the rationale behind their implementation, including the choice of prior distributions, the handling of parameter constraints, and the specific log-density formulations used in **Stan**.

The user has three ways to define their models. One is through a .txt file, using Python to read it and then pass it to the `'BayesianInferenceService'` class. Alternatively, the user can define it directly in the Python file. It is also possible to use one of the models provided by `stan_circular_inference`: `VonMisesMK`, `VonMisesM`, `Cardioid`, `WrappedCauchy`, and `Mixture`.

2.1. ProbabilisticModel class

This library provides an abstract class, denoted `ProbabilisticModel` (cf. appendix A), that enables the implementation of probabilistic models in Python using Stan code. If the user wishes to add additional distributions, they must implement the two abstract methods promised by the abstract class: the `gen_stan_model` and `get_parameters_prior` functions, as will be shown in the following sections.

2.2. VonMisesMK class

The class `VonMisesMK` represents the Von Mises distribution and aims to infer the parameters `mu` and `kappa` from the `values` (dataset) and `N` (size of the dataset) it receives. It directly uses the existing implementation of the Von Mises distribution in Stan (cf. appendix B). In the context of Bayesian inference, the `VonMisesMK` class estimates both μ and κ simultaneously, making it suitable for datasets where the concentration is unknown.

The Von Mises distribution is one of the most fundamental distributions in circular statistics. Its probability density function (pdf) is given by:

$$\theta \mapsto \frac{1}{2\pi I_0(\kappa)} \exp(\kappa \cos(\theta - \mu)),$$

where $\theta \in [0, 2\pi)$ is the angular variable, $\mu \in [0, 2\pi)$ is the mean direction, and $\kappa \geq 0$ is the concentration parameter. The function $I_0(\kappa)$ is the modified Bessel function of the first kind and order zero [Ley and Verdebout \(2017\)](#), defined as:

$$I_0(\kappa) = \frac{1}{2\pi} \int_0^{2\pi} \exp(\kappa \cos \theta) d\theta = \sum_{r=0}^{\infty} \frac{1}{(r!)^2} \left(\frac{\kappa}{2}\right)^{2r}.$$

The Bessel function acts as a normalizing constant, ensuring that the density integrates to one over the circle. For $\kappa = 0$, the Von Mises distribution reduces to the uniform circular distribution, while as $\kappa \rightarrow \infty$, it approaches a point distribution concentrated at μ . The Von Mises distribution is often referred to as the circular normal distribution, a term introduced by [Gumbel, Greenwood, and Durand \(1953\)](#), as demonstrated in Figure 2.

2.3. VonMisesM class

This is another implementation of the Von Mises distribution, adapted from BUGS as presented in [Neto \(2015\)](#). In Stan, it is not possible to perform the "zero trick" commonly used in BUGS to bypass missing log-density functions. However, this implementation (cf. appendix C) aims to infer only the parameter μ , assuming that the user knows the concentration parameter κ . The user must provide the dataset `values`, the number of samples N , and `kappa`.

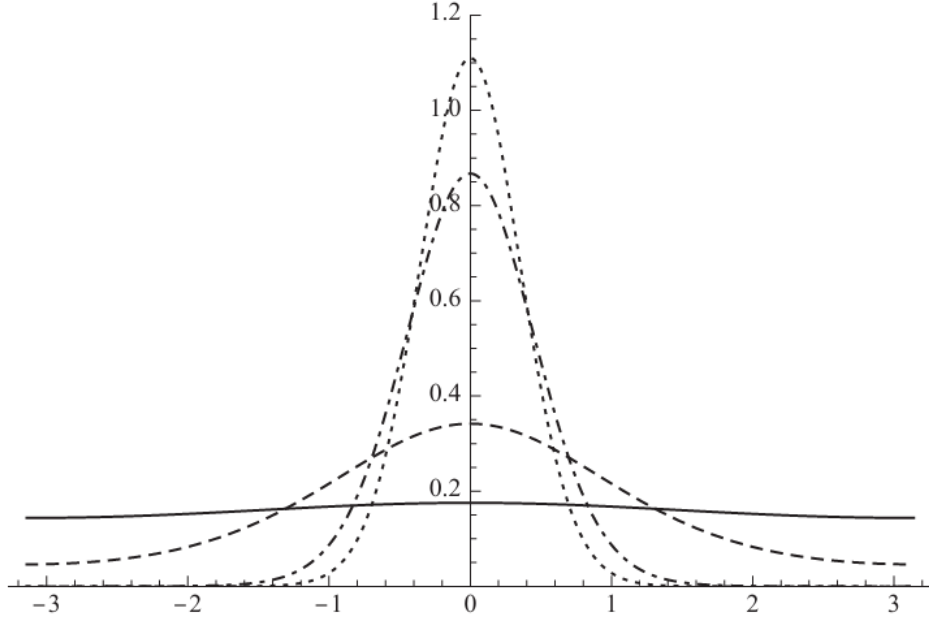


Figure 2: Von Mises densities with $\kappa = 0.1, 1, 5, 8$. [Mardia and Jupp \(2009\)](#)

This class is useful in scenarios where the concentration parameter is known a priori or has been estimated separately, allowing the user to focus exclusively on inferring the mean direction with reduced computational overhead.

2.4. Cardioid class

The `Cardioid` class represents the Cardioid distribution and aims to infer μ and ρ from the given dataset.

The Cardioid distribution is a circular distribution with density given by:

$$f(\theta \mid \mu, \rho) = \frac{1}{2\pi} (1 + 2\rho \cos(\theta - \mu)), \quad 0 \leq \rho < \frac{1}{2}.$$

Here, μ is the mean direction, and ρ controls the concentration, as displayed in Figure 3. The Cardioid distribution is exclusively suited for unimodal symmetric circular data and is particularly useful for modeling circular data with bounded support and moderate dispersion. The parameter ρ must lie in $[0, 0.5)$ to ensure non-negativity of the density. When $\rho = 0$, the distribution reduces to the uniform circular distribution.

To implement this distribution in `Stan`, we transform the density into logarithmic form, since `Stan` operates on the log scale for numerical stability. The log-density becomes:

$$\log f(\theta \mid \mu, \rho) = -\log(2\pi) + \log(1 + 2\rho \cos(\theta - \mu)).$$

This is the formulation implemented in `Stan` and presented in the appendix D.

2.5. WrappedCauchy class

The Cauchy distribution (also known as the Lorentz distribution) is the distribution of the

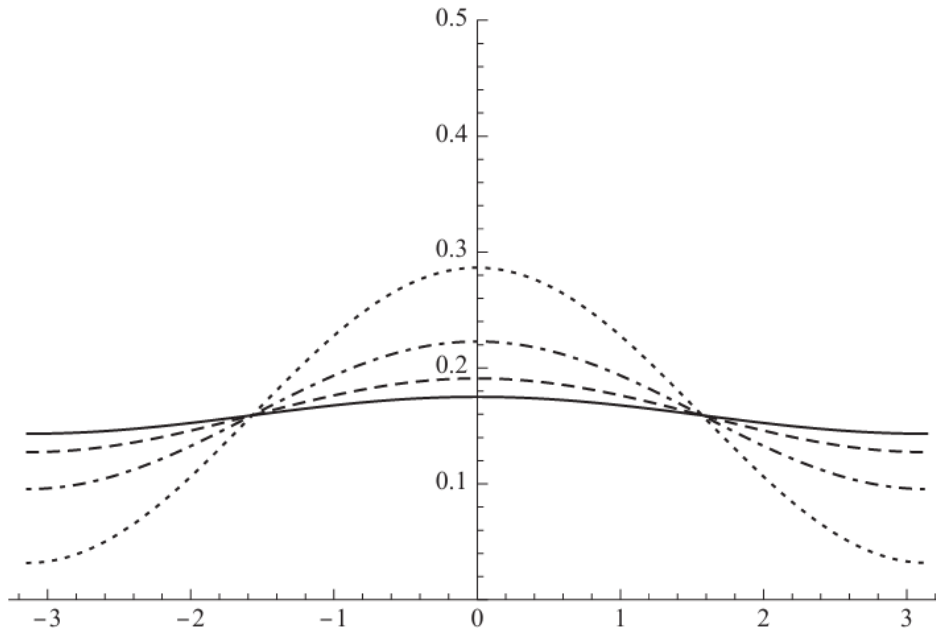


Figure 3: Cardioid densities with $\rho = 0.05, 0.1, 0.2, 0.4$. [Mardia and Jupp \(2009\)](#)

ratio of two Gaussians, and an extreme instance of a heavy-tailed distribution, where the mean and variance are not defined. The wrapped Cauchy distribution is obtained by taking a Cauchy distribution on the real line and wrapping it around the unit circle. Its density function is given by:

$$f(\theta \mid \mu, \rho) = \frac{1}{2\pi} \cdot \frac{1 - \rho^2}{1 + \rho^2 - 2\rho \cos(\theta - \mu)}, \quad 0 \leq \rho < 1,$$

where $\mu \in [0, 2\pi)$ is the location parameter (mean direction) and ρ is the concentration parameter, as illustrated in Figure 4. Unlike the Von Mises distribution, the wrapped Cauchy distribution with its heavier tails, makes it more robust to outliers. As $\rho \rightarrow 0$, the distribution converges to the uniform circular distribution; as $\rho \rightarrow 1$, it becomes increasingly concentrated around μ .

The `WrappedCauchy` class represents the wrapped Cauchy distribution and aims to infer μ and ρ from the given dataset.

To implement this distribution in `Stan`, we take the logarithm of the density, resulting in:

$$\log f(\theta \mid \mu, \rho) = -\log(2\pi) + \log(1 - \rho^2) - \log(1 + \rho^2 - 2\rho \cos(\theta - \mu)).$$

This formulation ensures numerical stability and its exact implementation is shown in appendix E. The wrapped Cauchy distribution is particularly well-suited for modeling circular data exhibiting pronounced clustering with heavy tails, where the von Mises distribution might underestimate the probability of extreme angular deviations.

2.6. Parameters

The uncertainty around the parameters for each of these distributions, in the Bayesian frame-

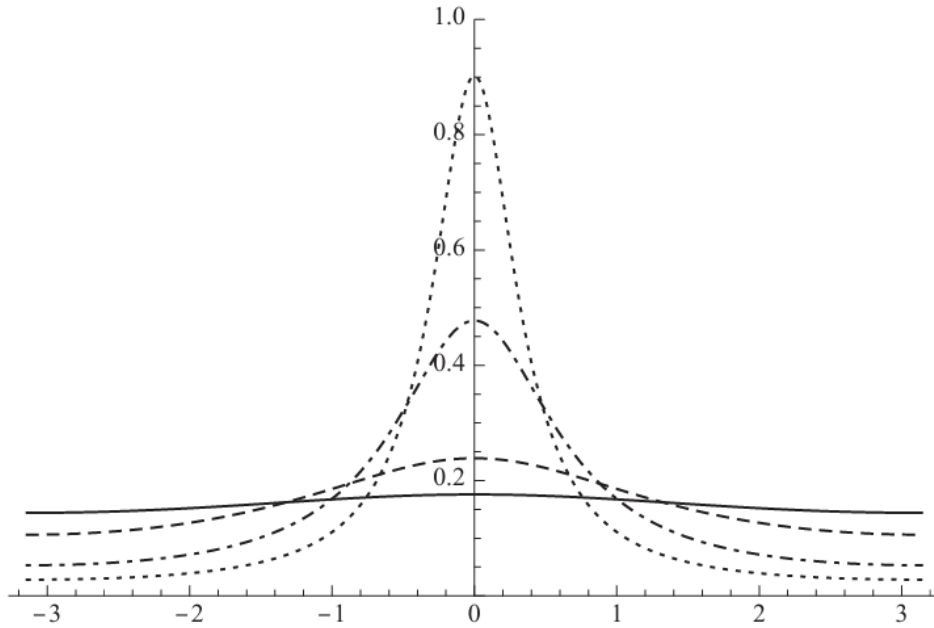


Figure 4: wrapped Cauchy densities with $\rho = 0.05, 0.2, 0.5, 0.7$. [Mardia and Jupp \(2009\)](#)

work, can also be modeled by prior distributions, usually called hyper-parameters. This package provides several options for the user to choose:

- Normal - implements the abstract class `MeanParameter`.
- Uniform - implements the abstract class `MeanParameter`.
- Bernoulli - implements the abstract class `MeanParameter`.
- Poisson - implements the abstract class `MeanParameter`.
- Gamma - implements the abstract class `VarianceParameter`.
- Exponential - implements the abstract class `VarianceParameter`.

The following section (`Mixture` class) illustrates the concrete use of some of these prior distributions, through the creation of a mixture model intended to demonstrate their respective usage.

2.7. Mixture class

The `Mixture` class represents a mixture model. This class receives two implementations of the abstract class `ProbabilisticModel` which, at present, can be one of the available circular distributions: `VonMisesMK`, `VonMisesM`, `Cardioid`, or `WrappedCauchy`.

The next code presents a complete workflow, showing how to prepare a `Mixture` instance that is then given to the inference service object that performs the inferential task:

```
# All required dependencies
from pycircstat2.distributions import wrapcauchy, cardioid
import numpy as np
from stan_circular_inference.factories.cardioid_factory import Cardioid
from stan_circular_inference.factories.wrapped_cauchy_factory import WrappedCauchy
from stan_circular_inference.factories.mixture_factory import Mixture
from stan_circular_inference.factories.distributions_factory import Normal, Uniform, \
    Exponential
from stan_circular_inference.service import BayesianInferenceService

# Configuration for generating samples from
# distributions to test the model via pycircstat2
size = 50
mu1, rho1 = np.pi / 4, 0.2
mu2, rho2 = 7/4 * np.pi, 0.7
samples1 = cardioid.rvs(size=size, mu=mu1, rho=rho1)
samples2 = wrapcauchy.rvs(size=size, mu=mu2, kappa=rho2)

# Prior distributions for the first component
mu1 = Uniform(0, 2*np.pi)
rho1 = Normal(0.25, 0.25)

# Prior distributions for the second component
mu2 = Uniform(0, 2*np.pi)
rho2 = Normal(0.5, 0.5)

# The first component follows a Cardioid distribution
dist1 = Cardioid(mu1, rho1)
# The second component follows a Wrapped Cauchy distribution
dist2 = WrappedCauchy(mu2, rho2)

# Constructs the mixture model combining the two components
model = Mixture(dist1, dist2)

# Initialises the Bayesian inference service with the mixture model
service = BayesianInferenceService(model)

# Concatenates samples from the two distinct distributions
samples = np.concatenate([samples1, samples2])

# Initialises the model data structure
data = {
    'N': len(samples),
    'values': samples
}

# Visualises the generated circular data
```

```

service.circular_graphic(samples, min_val=0, max_val=2*np.pi)

# Builds the Bayesian model
posterior = service.build_model(data)
# Performs 500 simulations in each of the four Markov chains
fit = service.get_samples(posterior, size*10)
# Extracts the posterior values for all specified parameters
# (mean, scale, and one sample from each mixture component)
values = service.get_values(fit, parameters=['mu1', 'rho1', 'mu2', 'rho2', \
      'mixing_weight.1', 'mixing_weight.51'])

# Retrieves model statistics as a pandas DataFrame
service.get_statistics(fit)

# Plots all specified parameters
service.multiple_graphics(values, param_names=['mu1', 'rho1', 'mu2', 'rho2', \
      'mixing_weight.1', 'mixing_weight.51'], min_val=0, max_val=2*np.pi, \
      parameters_type=[True, False, True, False, False, False])

```

Figure 5 presents an example of a dataset generated from a mixture of two von Mises distributions, with mean directions $\mu = \frac{\pi}{4}$ and $\mu = -\frac{\pi}{4}$, respectively.

The resulting inferred values, shown in Figure 6, are used to determine whether a given data point belongs to distribution 1 (von Mises with $\mu = \frac{\pi}{4}$) or to distribution 2 (von Mises with $\mu = -\frac{\pi}{4}$).

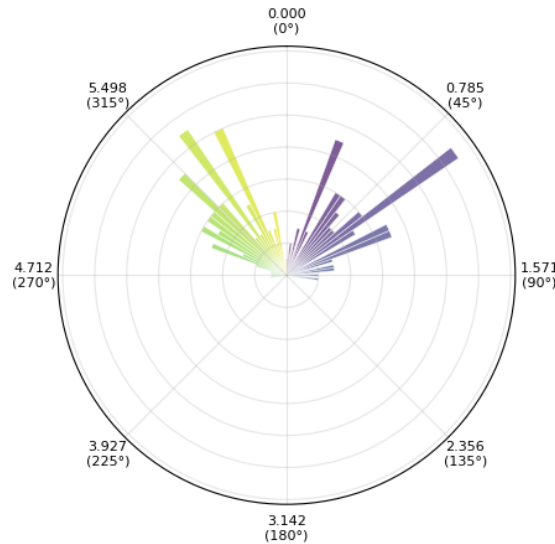


Figure 5: A dataset visualization from a mixture of two Von Mises distributions (with mu values, $\frac{\pi}{4}$ and $-\frac{\pi}{4}$, respectively).

The purpose of `Mixture` is to infer the location and spread parameters of the two distributions for each given mixture model, and the mixing weight of all data points. This mixing weight is a vector, where each element x_i takes values between zero and one, in order to indicate whether

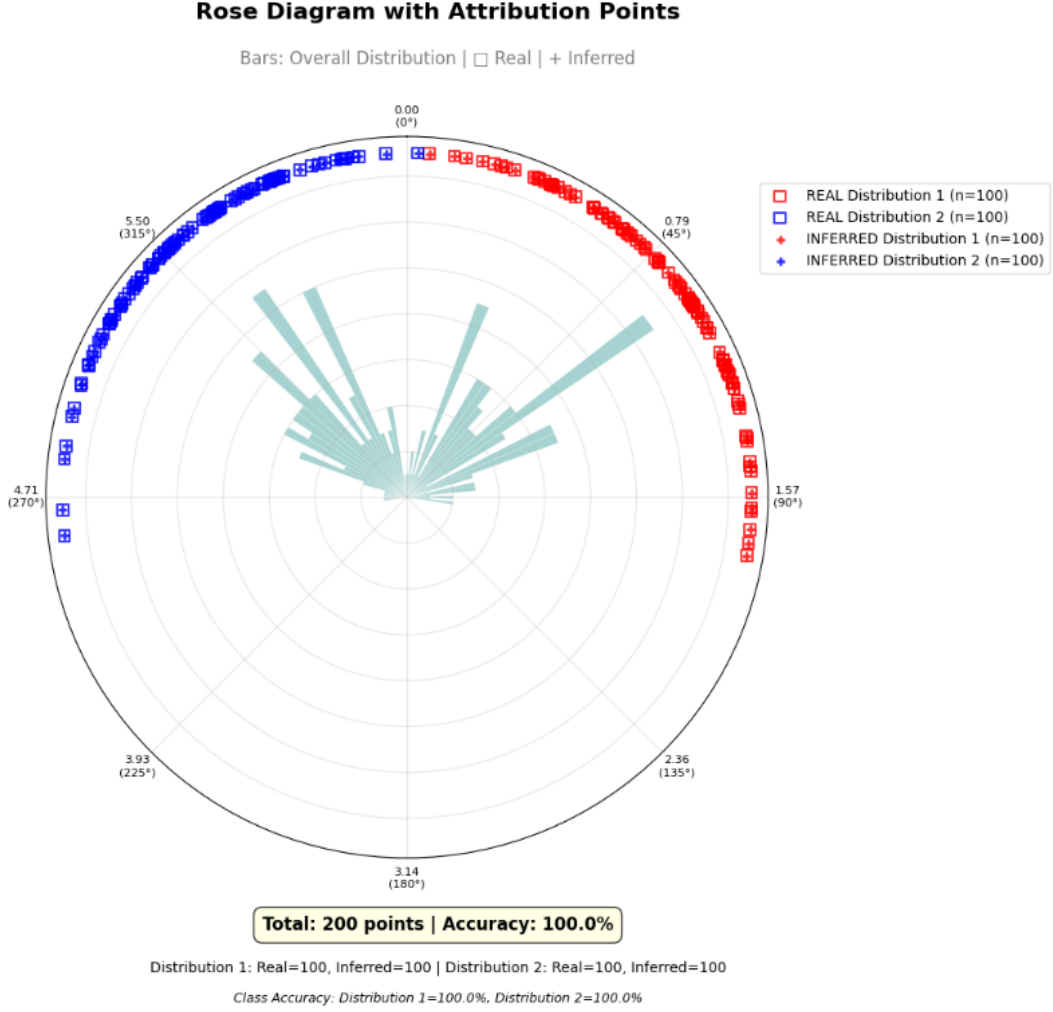


Figure 6: Inferred values from the mixture model present in Figure 5.

x_i belongs to the first distribution ($\text{mixing_weight}[i] < 0.5$) or the second distribution ($\text{mixing_weight}[i] \geq 0.5$).

For stability reasons, and because mixture models are considerably more complex than normal models, a limit was imposed on the concentration parameter of the Von Mises distribution, $0 \leq \kappa \leq 50$. In the simple model, numerical instability begins at $\kappa \geq 100$. However, in mixture models, there were samples where the estimation for the concentration κ diverged, which would discard that simulation, completely losing the computation performed.

3. Installation

This section will guide the user step by step so that no errors are encountered during the library installation process. Ideally, running a single command would suffice to have everything ready. However, one of the required files exceeds the maximum size supported by GitHub, which forces the user to install it manually. The GitHub page can be found in Almeida (2026).

3.1. Dependencies

`stan_circular_inference` requires a set of prerequisite packages to be present on the system. Below is the list of dependencies and the respective references.

- Python version 3.10.
- `setuptools` (version <81.0): The package for building and distributing Python packages.
- NumPy [Harris \(2020\)](#) (version 2.2.6): The fundamental scientific computing package; provides a multidimensional array type and many useful functions for numerical analysis.
- pandas [McKinney \(2010\)](#) (version 2.3.3): Data manipulation and analysis library, built on NumPy. Used to handle input data as DataFrame objects.
- SciPy [Virtanen \(2020\)](#) (version 1.15.3): Library of algorithms for mathematics, science, and engineering. Required for statistical functions and optimisations.
- ArviZ [Kumar \(2019\)](#) (version 0.22.0), optional: Exploratory analysis of Bayesian models. Provides plotting and diagnostics for posterior distributions (e.g., trace plots, R-hat).
- Matplotlib [Hunter \(2007\)](#) (version 3.10.7), optional: 2D plotting library that produces publication-quality figures in a variety of image formats and interactive environments.
- PyStan [Stan Development Team \(2024\)](#) (version 3.10.0): The Python interface to Stan, used for Bayesian inference on circular data.

3.2. Step by step

To obtain a stable environment for using the tool, the user is recommended to follow the steps below.

On Windows environments, execute the following commands:

```
> wsl --install (requires to restart)
> wsl -d Ubuntu
```

To obtain a stable version of Anaconda:

```
> sudo wget https://repo.continuum.io/archive/Anaconda3-2022.10-Linux-x86_64.sh
> bash Anaconda3-2022.10-Linux-x86_64.sh (requires to restart)
// Do you wish the installer to initialise Anaconda3? > yes
```

Close the command line and open it again.

On Windows systems, to enter the Linux subsystem:

```
> wsl -d Ubuntu
```

To install the Ubuntu dependencies:

```
> sudo apt update
> sudo apt install build-essential
> sudo apt-get install manpages-dev
```

To create an environment in Anaconda:

```
> conda create -n enviroment_name python=3.10
```

To activate that environment:

```
> conda activate enviroment_name
```

To install the Anaconda dependencies:

```
> conda install -c conda-forge gcc libstdcxx-ng
```

Finally, to install the tool's dependency:

```
> pip install stan-circular-inference
```

4. Pipeline

This section covers the provided pipeline, `BayesianInferenceService`, and shows how to create it as well as the available methods. At the end of this chapter, there is an example of its complete usage.

4.1. Data types

The library supports the following data types:

ANGLES the range of values is between 0 and 360

RADS the range of values is between 0 and 2π

HOURS the range of values is between 0 and 24

MONTHS the range of values is between 0 and 12

PERCENT the range of values is between 0 and 1

WEEK the range of values is between 1 and 7

WEEKS the range of values is between 1 and 52

MONTH the range of values is between 1 and 31

The method `circular_graphic` normalize the provided values according to the minimum and maximum value of each data type. Although percentage is not considered circular data, it is precisely used for displaying linear plots.

4.2. Building the model

The `build_model` function compiles the Stan model using the provided data dictionary. It relies on the `stan.build` function from the **PyStan** package. The function expects a dictionary data whose keys and types correspond exactly to the data variables declared in the Stan model code (e.g., integers, real numbers, arrays). The arguments is given in Table 1. For example, a typical model for circular data might require `'N': 20, 'values': np.array([...])`. The method returns a compiled Stan model object (`stan.Model`), which can then be used for posterior sampling via the `get_samples` function. If the compilation fails due to type mismatches or invalid constraints, appropriate exceptions (`TypeError`, `RuntimeError`) are raised with actionable suggestions.

Key	Type	Description
data	Dictionary	A dictionary where keys correspond to data variables declared in the Stan model (e.g., <code>'N': 20, 'values': np.array([...])</code>). Values must be of the appropriate types (integers, floats, arrays, etc.) as expected by the Stan model.

Table 1: A description of `build_model` function arguments.

4.3. Fitting the model

The `get_samples` method draws MCMC samples from the posterior distribution of a compiled Stan model. It internally calls the `sample` function of the `stan.Model` object, using the No-U-Turn Sampler (NUTS) implemented in **PyStan** package. The most important arguments are given in Table 2; the obligatory one is `posterior`. The method takes four main arguments: the compiled posterior model (`posterior` obtained from `build_model`), the number of samples to draw per chain (`sample_amount`, default 50000), the number of Markov chains (`n_chains`, default 4), and optional initial values for the parameters (`init`). The total number of posterior draws equals `n_chains * sample_amount`. If `init` is provided, it can be a single dictionary (same initial values for all chains) or a list of dictionaries (one per chain). If the value is `None`, Stan randomly generates initial values. The method returns a `StanFit` object containing the posterior samples, diagnostics, and metadata. If the compilation fails due to type mismatches or invalid constraints, appropriate exceptions (`RuntimeError`, `TimeoutError`) are raised with actionable suggestions.

4.4. Sampling values

The `get_values` method extracts posterior samples for specific parameters from a `StanFit` object (returned by `get_samples`). The method iterates over the chains stored in variable `fit.stan_outputs`, decodes each line (which is in JavaScript Object Notation (JSON) format), and collects the values corresponding to the requested parameter only from lines whose

Key	Type	Description
posterior	Dictionary	A dictionary where keys correspond to data variables declared in the Stan model (e.g., 'N': 20, 'values': np.array([...])). Values must be of the appropriate types (integers, floats, arrays, etc.) as expected by the Stan model.
sample_amount	Integer	Number of posterior samples to draw per chain after warmup. The total number of draws will be n_chains * sample_amount. Default value is 50000.
n_chains	Integer	Number of chains used to draw samples. Default value is 4
init	Any	Initial values for the parameters. Can be: • Dictionary - A single dictionary (same initial values for all chains). • List - A list of dictionaries (one per chain). • None (default): Stan randomly generates initial values.

Table 2: A description of `get_samples` function arguments.

topic is "sample" and that contain a dictionary of values. The result is a dictionary where each key is a parameter name and each value is a list of samples (ordered in the sequence they appear across all chains). To access individual elements of a vector (e.g., `theta`), use dot notation followed by the index, starting at 1 (e.g., `theta.1`, `theta.3`). This method is useful for obtaining raw samples of specific parameters, which can then be used for diagnostics, custom calculations, or visualisations (such as `circular_graphic`).

4.5. Statistics summary

The `get_statistics` function computes a summary statistics table from a fitted PyStan model, with automatic handling of circular variables. The method first converts the PyStan fit object into an **ArviZ InferenceData** object, then identifies circular variables by selecting all parameter names that start with `mu` (e.g., `mu1` in mixture models). Using ArviZ's `summary` function, it generates default statistics for all parameters, including the mean, standard deviation, effective sample size, and $\hat{R}$, while also specifying the circular variables and the confidence interval level for the Highest Density Interval (HDI), given by the user or its default value, 89, due to a reference of [Davidson-Pilon \(2015\)](#). one example of a possible table is present in Figure 7.

For circular variables, the method adjusts the mean and HDI bounds by wrapping them modulo 2π to ensure that the values remain within the interval $[0, 2\pi)$. The resulting summary is printed and returned as a **pandas DataFrame**. This method is useful for obtaining a concise and interpretable summary of posterior distributions.

Key	Type	Description
fit	StanFit	A Stan fit object returned by <code>get_samples</code> . This object has the attribute <code>stan_outputs</code> , a list of bytes (one per chain) containing the raw sampler output.
parameters	List<String>	List of parameter names to extract. Names must match exactly those used in the Stan model. To access vector elements, use dot notation with the index (starting at 1). Examples: • "mu" – scalar parameter. • "theta.1" – first element of vector <code>theta</code>. • "mixing_prop.2" – second element of vector <code>mixing_prop</code>.

Table 3: Description of the `get_values` function arguments.

	mean	sd	hdi_44.5%	hdi_55.5%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
mu	4.760295	0.03967	4.753185	4.763865	0.00013	0.00020	32932.05180	26573.91368	1.00009
kappa	0.091950	0.00367	0.091860	0.092850	0.00002	0.00002	35149.64374	26556.56183	1.00023

Figure 7: Example summary statistics from the posterior inference of the μ and κ parameters for the wind dataset.

Key	Type	Description
fit	PyStan fit object	A fitted PyStan model object containing posterior samples. This object is typically the result of calling <code>sampling()</code> on a Stan model.
confidence_interval	Integer	Confidence level (in percent) used for the Highest Density Interval (HDI). Default is 89. The HDI bounds are calculated as the lower and upper percentiles corresponding to $(100 - \text{confidence_interval})/2$ and $\text{confidence_interval} + (100 - \text{confidence_interval})/2$, respectively.

Table 4: Description of the `get_statistics` function arguments.

4.6. Visualization

To help users visualise inference results, several methods are available for different use cases. All circular plots share the same polar configuration: 0° at the top, clockwise direction, and tick labels every 45° showing both the raw value and the corresponding angle in degrees. Data are normalised to the interval $[0, 2\pi)$ using `normalize_values` based on the given `data_type` (radians or degrees). The visible angular range can be restricted by `min_val` and `max_val`, if omitted, the full data range is used.

The `circular_graphic` method generates a single circular (polar) histogram or a smooth density plot for a set of angular data. From a dataset generated from a von Mises distribution with mean direction $\mu = \pi$, the results are presented in Figure 8 and Figure 9.

- **Histogram mode** (`density=False`): draws a rose diagram (bars/wedges) with frequencies, coloured by the Viridis colormap. The radial limit is set to $1.1 \times \max(\text{frequencies})$.
- **Density mode** (`density=True`): fits a Gaussian KDE, normalises the curve to a peak height of 1, and fills the area under the curve. The radial axis is fixed to $[0, 1.1]$.

The plot title is “Diagrama de rosas”. The method calls `plt.show()` and returns nothing.

Key	Type	Description
<code>interest_parameter_values</code>	list or <code>ndarray</code>	Raw angular data to be visualised.
<code>n_intervals</code>	Integer	Number of equal-width bins (or interpolation points) used to discretise the circular domain. Default is 100.
<code>density</code>	Boolean	If <code>True</code> , a smooth kernel density estimate (KDE) is plotted; if <code>False</code> (default), a circular bar chart (rose diagram) is drawn.
<code>min_val, max_val</code>	float or <code>None</code> optional	Lower and upper bounds of the angular range to display. If <code>None</code> , the full range of the data (after normalisation) is used. Setting these values allows zooming into a specific sector of the circle (e.g., 40° to 60° instead of the full 360°).
<code>data_type</code>	<code>DataType</code> enum	Specifies the unit of the input data. The method uses <code>data_type.value[0]</code> and <code>data_type.value[1]</code> to normalise the values to the target circular interval (typically $[0, 2\pi)$). Default is <code>DataType.RADS</code>

Table 5: Description of the `circular_graphic` method arguments.

The `match_points_to_distributions` function overlays ground-truth and inferred cluster assignments on a circular histogram. Designed for evaluating mixture models or binary classification where each point has a true label (0 or 1) and a predicted label. The Figure 10 combines three layers:

1. A rose diagram (bars) showing the overall angular distribution.
2. **Unfilled squares** representing the true attribution (edge colours from `colors`).
3. **Tight plus signs** representing the inferred attribution (filled with the corresponding colour).

Markers are placed at a fixed radius just above the highest bar. The method also computes and displays overall accuracy, confusion matrix, per-class accuracies, and point counts. It returns a dictionary with these statistics.

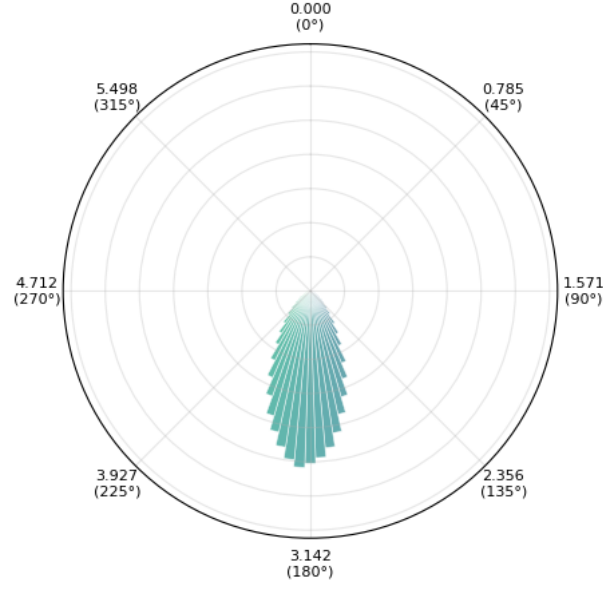


Figure 8: Rose Diagram with data generated from a VonMises with $\mu = \pi$.

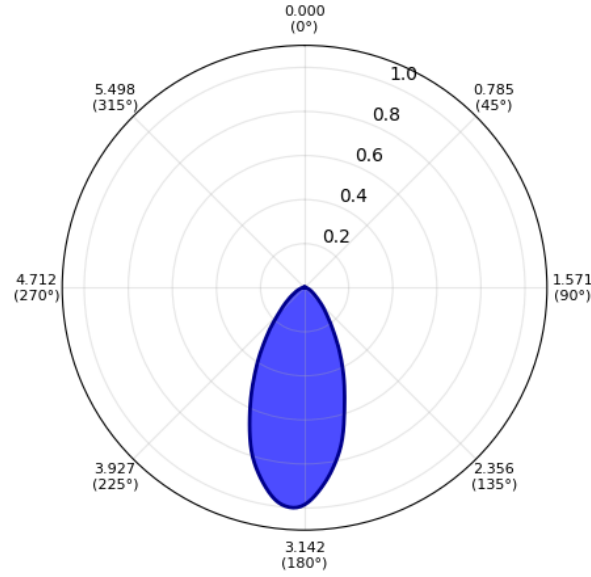
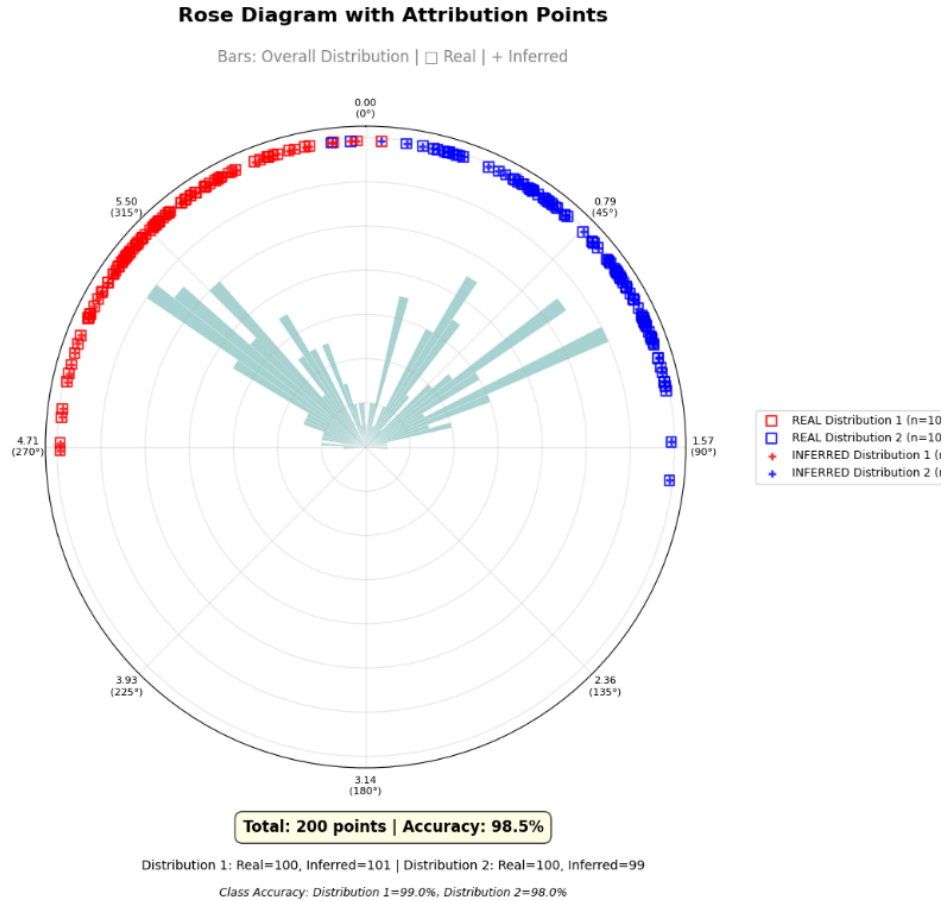


Figure 9: Probability density with data generated from a VonMises with $\mu = \pi$.

The `multiple_graphics` method generates a grid of plots for several parameters simultaneously, as shown in Figure 11. It supports both circular (polar) and linear visualisations, determined by the `parameters_type` list. This method is particularly useful for inspecting posterior distributions of multiple variables (e.g., mixture components) in a compact, organised layout. The function automatically arranges the plots in up to 3×3 subplots per figure, saving each full figure as a PNG file (named `image1.png`, `image2.png`, ...) and displaying it interactively by calling `plt.show()`. To avoid file overwriting, the user must save the PNG files to a different location before re-executing the program that calls this method, as the

Key	Type	Description
<code>values</code>	list or ndarray	Angular data values (in the unit specified by <code>data_type</code>).
<code>real_attribution</code>	list of int	Ground-truth labels (0 or 1) for each point.
<code>inferred_attribution</code>	list of int	Predicted labels (0 or 1) for each point.
<code>n_intervals</code>	int	Number of bins for the circular histogram. Default is 100.
<code>min_val, max_val</code>	float or None, optional	Lower and upper bounds of the angular range to display. If None, the data min and max are used.
<code>data_type</code>	DataType enum	Unit of the input data; used for normalisation to $[0, 2\pi)$. Default is radians (<code>DataType.RADS</code>).

Table 6: Description of the `match_points_to_distributions` method arguments.Figure 10: Matched points in a mixture model of two Von Mises distributions (with μ values, $\frac{\pi}{4}$ and $-\frac{\pi}{4}$, respectively).

method will overwrite the files with the most recent results.

Key	Type	Description
interest_parameter_values	Dictionary	A dictionary where keys are parameter names and values are the corresponding data arrays.
n_intervals	Integer	Number of bins (or interpolation points) used for each circular subplot. Default is 100.
density	Boolean	If True , a smooth KDE is plotted instead of bars for circular subplots; ignored for linear subplots. Default is False .
min_val, max_val	Any, optional	Lower and upper bounds for each parameter. • None – use data min/max. • Scalar – same bound applied to all parameters. • List – one bound per parameter (must match number of parameters). • Dictionary - maps parameter names to bounds.
param_names	List<String>, optional	Names of the parameters to be used in plot titles. Defaults to the keys of <code>interest_parameter_values</code> .
parameters_type	List<Boolean>	Indicates whether each parameter is circular (True) or linear (False). If the list length does not match the number of parameters, all are treated as circular. Default is considering every parameter circular.

Table 7: Description of the `multiple_graphics` method arguments.

4.7. Mixture models methods

One method for mixture models, `match_points_to_distributions`, has already been mentioned. Additionally, `get_mixture_statistics` computes the overall accuracy, per-distribution accuracy, and confusion matrix, while the helper `show_mixture_statistics` pretty-prints the output of `get_mixture_statistics`, the Figure 12 presents an example.

The `get_mixture_statistics` function computes classification accuracy and confusion matrix statistics from the true and inferred labels. After validating that all input arrays have equal length, it normalises the `values` to the target range (e.g., $[0, 2\pi]$ for `DataType.RADS` or $[0, 1]$ for `DataType.PERCENT`) and filters the observations that fall within `[min_val, max_val]`. The overall accuracy is defined as the percentage of correctly labelled observations among the filtered set. The per-class accuracies are:

- **Accuracy for class 0** (distribution 1): $\frac{TP}{TP+FP} \times 100$, where TP = true positives (real 0 and inferred 0), FP = false positives (real 0 but inferred 1).
- **Accuracy for class 1** (distribution 2): $\frac{TN}{TN+FN} \times 100$, where TN = true negatives (real 1 and inferred 1), FN = false negatives (real 1 but inferred 0).

The method returns a dictionary containing the overall accuracy, per-class accuracies, and

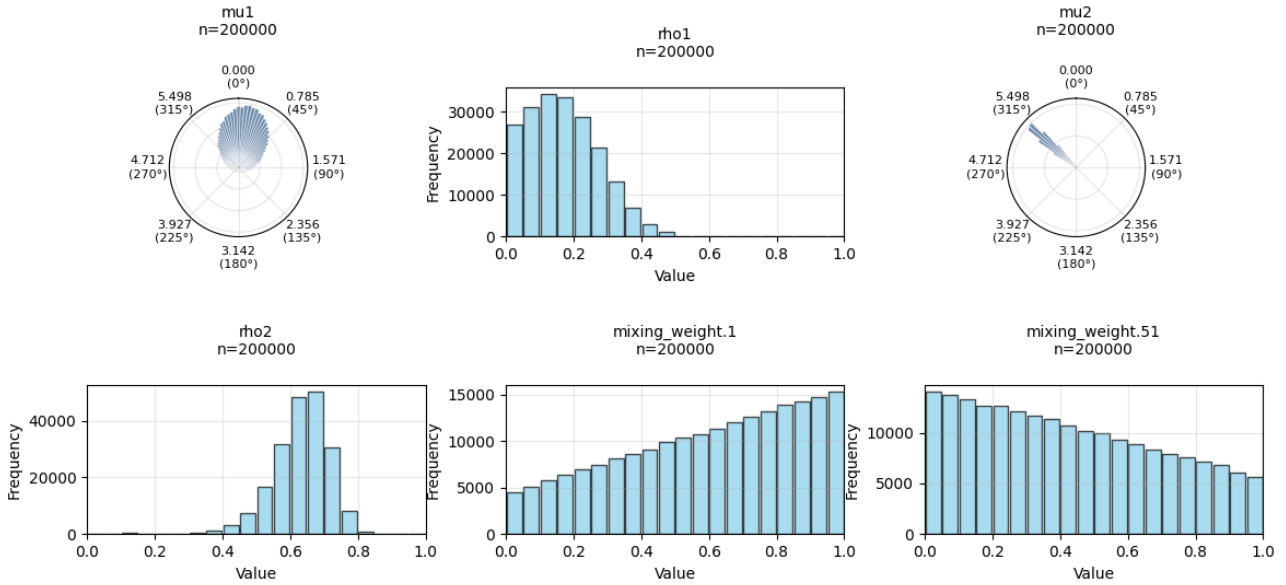


Figure 11: Multiple graphical visualization of a mixture model: Top Left - circular graphic of the μ_1 parameter; Top Center - linear graphic of the ρ_1 parameter (since $0 \leq \rho < 1$); Top Right - circular graphic of the μ_2 parameter; Bottom Left - linear graphic of the ρ_2 parameter; Bottom Center - linear graphic presenting the uncertainty that data point 1 corresponds to distribution 1 (it belongs to distribution 1 if the mean of the inferred values is < 0.5); Bottom Right - linear graphic presenting the uncertainty that data point 51 corresponds to distribution 1.

```

Accuracy: 100.0
Distribution_1_Accuracy: 100.0
Distribution_2_Accuracy: 100.0
Confusion Matrix:
-----| Predicted Values
Actual Values | 100 | 0
              | 0 | 100

```

Figure 12: Confusion matrix example.

the confusion matrix counts (TP , FN , FP , TN).

4.8. Missing data

In real-world data, it is common to encounter missing data, and there are various formats depending on how different individuals fill them out, such as Null, None, -1, -99999, etc. This typically depends on the domain of the sample. This package does not process missing data. So, unless some previous pre-processing occurs, the user should remove these data points.

4.9. Example

An example of using the tool on the dataset [SINTEF \(2025\)](#) is found where wind measurements are taken at five different positions. The interest in this case is the wind direction, and therefore the focus is on the values found in the column `wind_dir`.

```
# Import pandas for data manipulation
import pandas as pd

# Import NumPy for numerical array operations and mathematical functions
import numpy as np

# Load wind direction dataset from CSV file `wind_dataset`
df = pd.read_csv('datasets/wind_dataset.csv')

# Define uniform prior for mean direction mu on [0, 2*pi[
mu = Uniform(0, 2*np.pi)

# Define exponential prior for concentration kappa
kappa = Exponential(0.1)

# Instantiate the Von Mises model with given priors
model = VonMisesMK(mu, kappa)

# Extract wind direction values as float array
values = np.array(df['wind_dir'], dtype='float')

# Prepare data dictionary for Stan model
model_data = {'N': len(values), 'values': values}

# Create inference service with the model
service = BayesianInferenceService(model)

# Run MCMC sampling, obtain posterior samples for mu and kappa
interest_parameter_values = service.get_pystan_statistics(data=model_data, \
    parameters=['mu', 'kappa'], sample_amount=10000)

# Plot circular histogram/density for mu posterior
service.circular_graphic(interest_parameter_values['mu'], min_val=0, \
    max_val=2*np.pi)
```

5. Label Switching

This problem was encountered during testing and validation of the mixture models using the function `match_points_to_distribution`, where the result presented for a case in which the expected accuracy was close to 100% exhibited 0% accuracy, as can be observed in [Figure 13](#).

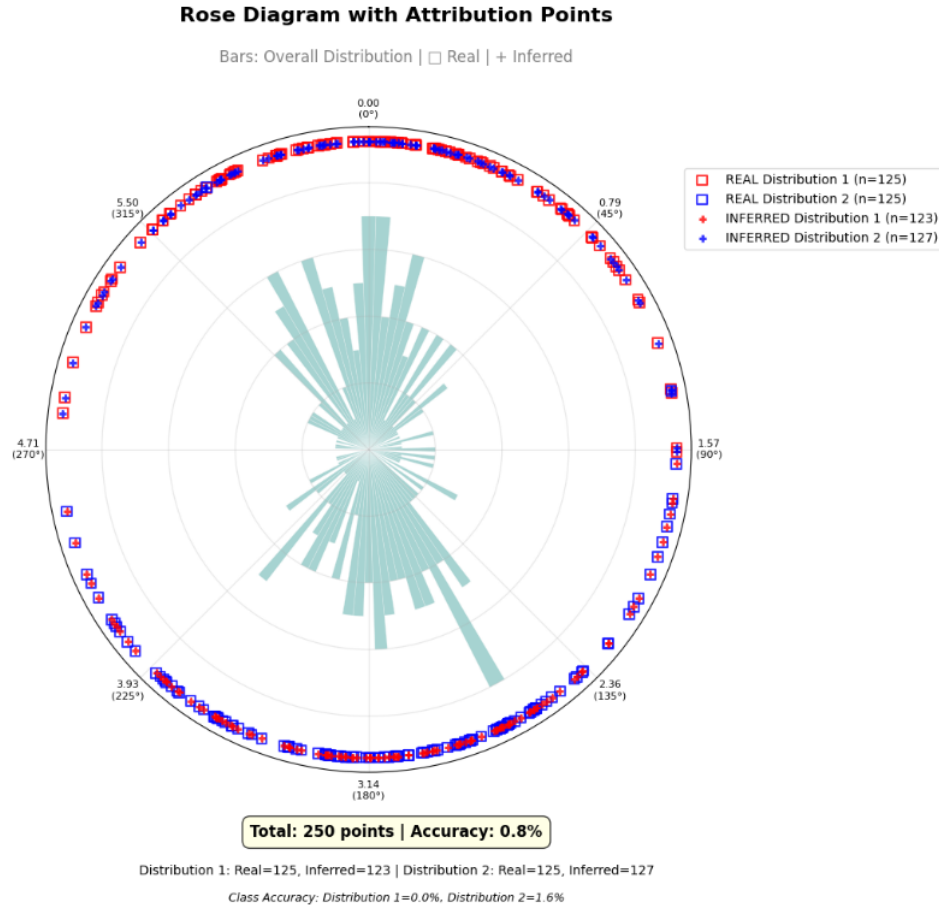


Figure 13: Label switching example.

After conducting research on this topic, a well-known issue in the field of Bayesian inference, three solutions were identified.

The first solution would be to specify that the mean value of distribution 1, μ_1 , is less than the mean value of distribution 2, μ_2 . However, if the correct μ_1 was, for example, 0, it could take negative values during the inference process. After normalization between 0 and 2π , in most cases, μ_1 would then be greater than μ_2 , which would invalidate the initial condition.

The second solution would consist, in cases of low accuracy, of swapping all points upon realizing that the values obtained from the inference result had been assigned to the wrong distribution. As a result, the model's accuracy would theoretically never fall below 50%, which would be incorrect.

The third solution is to apply no treatment at all and allow the user to choose how to address this problem. This is the chosen approach, given the lack of a general satisfying approach to this problem. This same conclusion is reached in [Thompson \(2014\)](#).

6. Model testing and validation

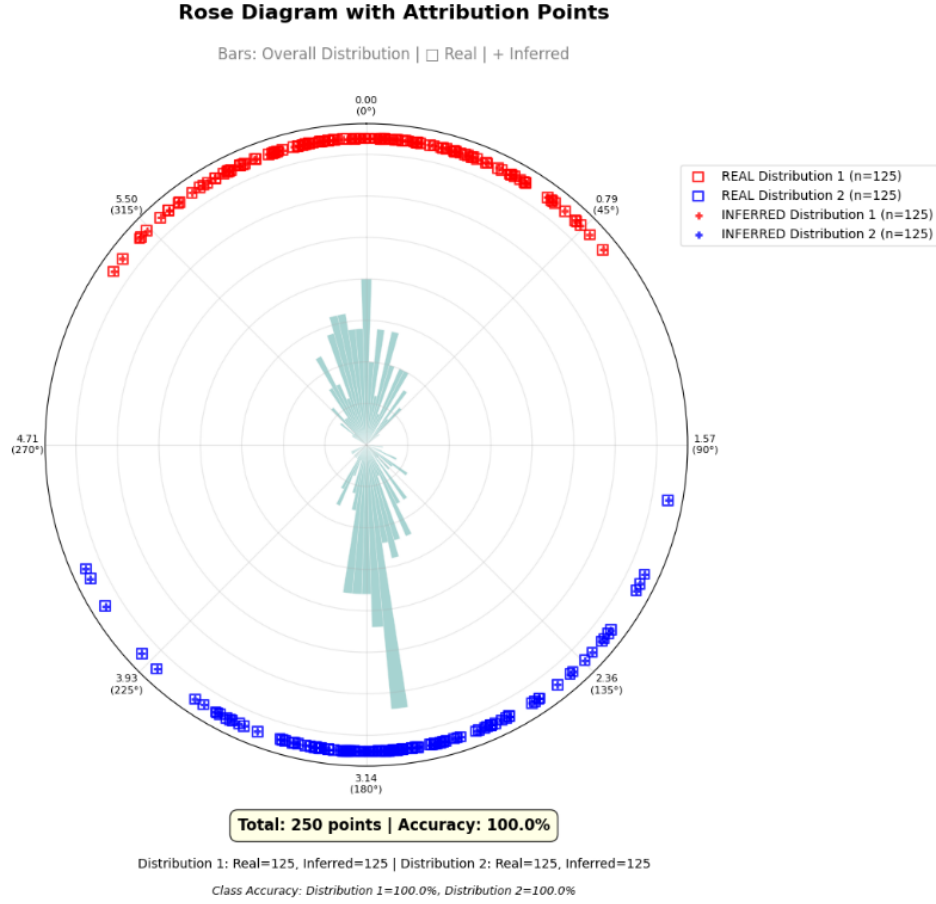


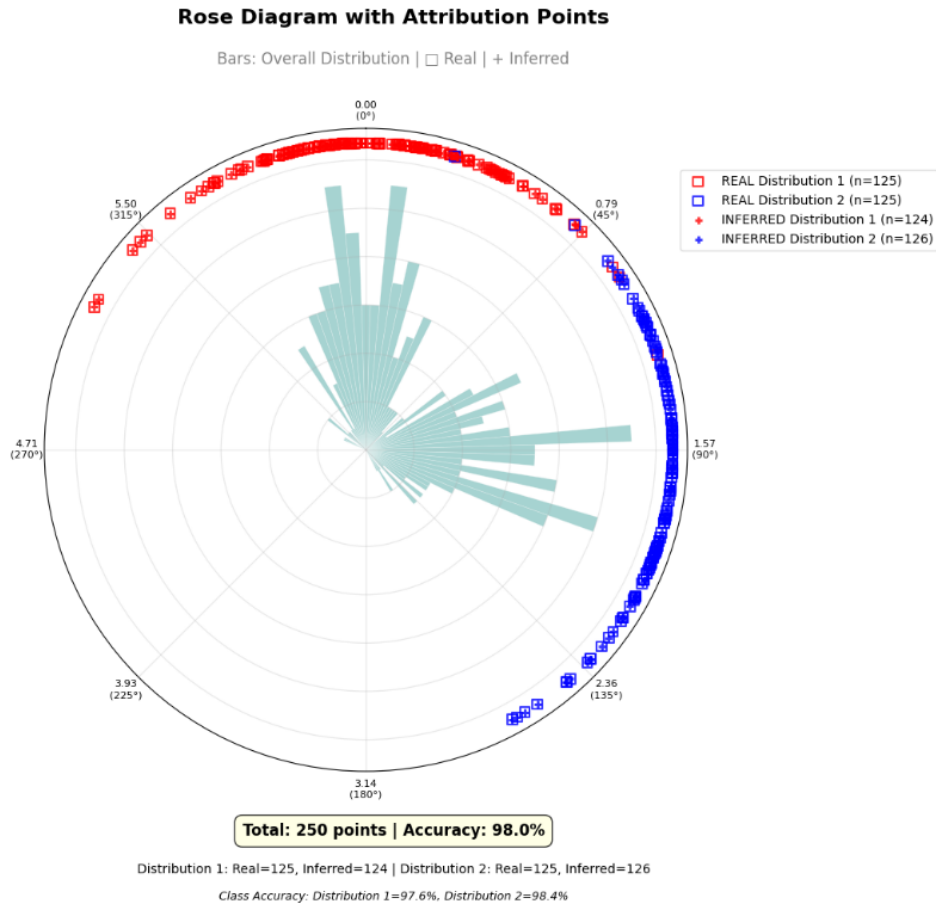
Figure 14: Testing the μ values to the furthest extent possible.

6.1. Parameter Inference

All models provided by the library were subjected to the following testing procedure. For each implemented distribution (von Mises, Cardioid, and wrapped Cauchy), 100 distinct datasets were generated, each containing 20 observations. For every dataset, a random mean direction μ and a random scale parameter (κ for the von Mises distribution, or ρ for the Cardioid and wrapped Cauchy distributions) were drawn. Subsequently, 10 000 simulation iterations were performed per dataset. After inference, we verified whether the true μ and the true scale parameter fell within the 95% credible (or confidence) interval obtained from the posterior samples. Across the 100 datasets, the procedure was considered successful if at least 95 of them contained both true parameters within their respective intervals, as expected under correct calibration of the 95% interval.

6.2. Model degradation

To assess the degradation of model performance under limited computational resources, multiple datasets were generated in advance, and a varying number of simulation iterations was employed for each dataset to verify that fewer iterations would lead to poorer inference outcomes. For each of the 100 distinct datasets, each containing 20 observations,

Figure 15: Another test in which the μ values converge.

three inference runs were performed with 10, 100, and 10 000 iterations, respectively. As expected, the proportion of times the true value lay inside the 95% confidence interval of the inferred values was monotonic with respect to the number of iterations, satisfying $\text{coverage}_{10} \leq \text{coverage}_{100} \leq \text{coverage}_{10000}$. The results can be consulted in Table 8.

Table 8: Coverage of the 95% confidence interval for different distributions

Distribution	10 iterations	100 iterations	10000 iterations
von Mises	0.88	0.95	0.95
Cardioid	0.94	0.96	0.98
wrapped Cauchy	0.85	0.92	0.92

Note: Values are expressed as percentages (e.g., 0.88 = 88%).

95% confidence interval for the inferred values.

6.3. Mixture models accuracy

For the purpose of testing the accuracy of the mixture models, the tests performed consisted of all combinations of the available models, where μ_1 (mean value of distribution 1) was always 0 and μ_2 (mean value of distribution 2) varied ($\mu_2 = \pi, \pi/2, \pi/4, 0$), for different scale values,

as exemplified in Figure 14 and Figure 15. Considering the label switching phenomenon, the results are positive: the mixture model correctly inferred the distribution to which each data point belongs, except when the two mean directions μ overlapped. In that overlapping case, the prediction accuracy of the model exhibited substantial fluctuation, as expected, since there are no way to distinguish between the two distributions.

7. Conclusion

The developed tool reduces the user’s required prior knowledge of a GPL programming language, enabling them to concentrate on specifying statistical models via the DSL, and interpreting the results of the inference process. This lowers the learning curve, allowing users to better exploit the package’s capabilities. Furthermore, it addresses a previously identified gap in Bayesian inference tools, particularly those dedicated to circular data analysis.

Probabilistic programming has proven to be a powerful paradigm for estimating circular distributions, offering remarkable flexibility in formulating complex statistical questions about parameters of interest. It provides an interesting alternative to traditional methods by facilitating the specification of sophisticated models that can describe circular data—using either single distributions or mixtures thereof—and adapt to intricate patterns.

The tool also provides mixture models that are both accurate and straightforward to create, supporting all combinations of the available model components.

A promising direction for future work is the incorporation of non-parametric probabilistic models, which would eliminate the need to prespecify a parametric circular distribution and instead infer the density directly from the data, providing greater robustness to model misspecification and revealing previously hidden structure in directional observations.

Acknowledgments

This work was supported by FCT through LASIGE Centro de Investigação em Ciência e Engenharia da Computação, ref. UID/00408/2025, DOI 10.54499 (https://doi.org/10.54499/UID/00408/2025), and the LASIGE Research Unit, ref. UID/408/2025.

References

- Abril-Pla O, Andreani V, Carroll C, Dong L, Fonnesebeck CJ, Kochurov M, Kumar R, Lao J, Luhmann CC, Martin OA, Osthege M, Vieira R, Wiecki T, Zinkov R (2023). “PyMC: A Modern and Comprehensive Probabilistic Programming Framework in Python.” *PeerJ Computer Science*, **9**(e1516). doi:10.7717/peerj-cs.1516.
- Albahnsen C (2019). “pycircular: A Python library for circular statistics.” URL <https://github.com/albahnsen/pycircular>.
- Almeida D (2026). “Stan Circular Inference.” GitHub repository. URL <https://github.com/wartuga/PEI>.

- circstat (2024). “pycircstat2: Circular Statistics with Python.” URL <https://github.com/circstat/pycircstat2>.
- Davidson-Pilon C (2015). *Bayesian Methods for Hackers: Probabilistic Programming and Bayesian Inference*. Addison-Wesley Professional. ISBN 978-0133902839. URL <https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>.
- Gumbel EJ, Greenwood JA, Durand D (1953). “The Circular Normal Distribution: Theory and Tables.” *Journal of the American Statistical Association*, **48**(261), 131–152. doi:10.1080/01621459.1953.10483462. <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1953.10483462>, URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.1953.10483462>.
- Hammer Ø, Harper DAT, Ryan PD (2020). “PAST: Palaeontological STatistics software package for education and data analysis.” URL <https://past.en.lo4d.com/windows>.
- Harris Cea (2020). “Array programming with NumPy.” *Nature*, **585**, 357–362. doi:10.1038/s41586-020-2649-2. URL <https://numpy.org/doc/>.
- Hunter JD (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, **9**(3), 90–95. doi:10.1109/MCSE.2007.55. URL <https://matplotlib.org/>.
- Kovach Computing Services (2025). “Oriana: Circular Statistics Software.” URL <https://www.kovcomp.co.uk/oriana/oribroc.html>.
- Kumar Rea (2019). “ArviZ a unified library for exploratory analysis of Bayesian models in Python.” *Journal of Open Source Software*, **4**(33), 1143. doi:10.21105/joss.01143. URL <https://arviz-devs.github.io/arviz/>.
- Ley C, Verdebout T (2017). *Modern directional statistics*. Chapman and Hall/CRC.
- Mardia KV, Jupp PE (2009). *Directional statistics*. John Wiley & Sons.
- McKinney W (2010). “Data Structures for Statistical Computing in Python.” In S van der Walt, J Millman (eds.), *Proceedings of the 9th Python in Science Conference*, pp. 56–61. URL <https://pandas.pydata.org/docs/>.
- Neto JP (2015). “Circular Statistics in R.” URL <https://jpneto.github.io/r/circular>.
- SINTEF (2025). “Processing : EddyPro Raw Output.” URL <https://data.sintef.no/feature/fe-000d522b-9de6-47c0-a615-03cfd3a0fe53>.
- Stan Development Team (2024). *PyStan: The Python Interface to Stan*. Versão 3.10.0, URL <https://pystan.readthedocs.io/>.
- Thompson J (2014). “Label Switching.” Bayesian Analysis with Stata (blog). URL <https://staffblogs.le.ac.uk/bayeswithstata/2014/05/18/label-switching/>.
- Virtanen Pea (2020). “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods*, **17**, 261–272. doi:10.1038/s41592-019-0686-2. URL <https://docs.scipy.org/doc/scipy/>.

A. ProbabilisticModel class

```

from abc import ABC, abstractmethod
from typing import Dict, Any

class ProbabilisticModel(ABC):

    def __init__(self, name: str):
        self.name = name
        self.parameters = {}

    @abstractmethod
    def gen_stan_model(self) -> str:
        """Return Stan code of the model"""
        pass

    @abstractmethod
    def get_parameters_prior(self) -> Dict[str, Any]:
        """Return the parameters distributions"""
        pass

```

B. VonMisesMK class

```

from stan_circular_inference.factories.model_factory import ProbabilisticModel
from stan_circular_inference.factories.parameters_factory import MeanParameter, \
    VarianceParameter
from typing import Dict, Any

class VonMisesMK(ProbabilisticModel):
    def __init__(self, mu: MeanParameter, kappa: VarianceParameter):
        super().__init__('Von Mises')
        self.mu = mu
        self.kappa = kappa

    def gen_stan_model(self) -> str:
        model_code = f"""
            data {{
                int<lower=0> N;
                vector[N] values;
            }}

            parameters {{
                real<lower=0, upper=2*pi()> mu;
                real<lower=0> kappa;

```

```

    }}

    model {{
        mu ~ {self.mu.get_code()};
        kappa ~ {self.kappa.get_code()};

        values ~ von_mises(mu, kappa);
    }}
"""

return model_code

def get_parameters_prior(self) -> Dict[str, Any]:
    return {
        'mu': self.mu,
        'kappa': self.kappa
    }

```

C. VonMisesM class

```

class VonMisesM(ProbabilisticModel):
    def __init__(self, mu: MeanParameter):
        super().__init__("Von Mises")
        self.mu = mu

    def gen_stan_model(self) -> str:
        model_code = f"""
            data {{
                int<lower=1> N;
                array[N] real values;
                real<lower=0> kappa;
            }}
            parameters {{
                real<lower=0, upper=2*pi()> mu;
            }}
            model {{
                real normalizer = log(2 * pi() * \
                    modified_bessel_first_kind(0, kappa));

                for (n in 1:N) {{
                    target += kappa * cos(values[n] - mu) - normalizer;
                }}

                mu ~ uniform(0, 2*pi());
            }}
        """

```

```

"""

return model_code

def get_parameters_prior(self) -> Dict[str, Any]:
    return {
        'mu': self.mu
    }

```

D. Cardioid class

```

from stan_circular_inference.factories.model_factory import ProbabilisticModel
from stan_circular_inference.factories.parameters_factory import MeanParameter, \
    VarianceParameter
from typing import Dict, Any

class Cardioid(ProbabilisticModel):
    def __init__(self, mu: MeanParameter, rho: VarianceParameter):
        super().__init__("Cardioid")
        self.mu = mu
        self.rho = rho

    def gen_stan_model(self) -> str:
        model_code = f"""
            data {{
                int<lower=0> N;
                vector[N] values;
            }}

            parameters {{
                real<lower=0, upper=2*pi()> mu;
                real<lower=0, upper=0.5> rho;
            }}

            model {{
                mu ~ {self.mu.get_code()};
                rho ~ {self.rho.get_code()};

                for (n in 1:N){{
                    target += -log(2*pi()) + log1p(2*rho*cos(values[n] - mu));
                }}
            }}
        """

        return model_code

```

```
def get_parameters_prior(self) -> Dict[str, Any]:
    return {
        'mu': self.mu,
        'rho': self.rho
    }
```

E. WrappedCauchy class

```
from stan_circular_inference.factories.model_factory import ProbabilisticModel
from stan_circular_inference.factories.parameters_factory import MeanParameter, \
    VarianceParameter
from typing import Dict, Any

class WrappedCauchy(ProbabilisticModel):
    def __init__(self, mu: MeanParameter, rho: VarianceParameter):
        super().__init__("Wrapped Cauchy")
        self.mu = mu
        self.rho = rho

    def gen_stan_model(self) -> str:
        model_code = f"""
            data {{
                int<lower=0> N;
                vector[N] values;
            }}

            parameters {{
                real<lower=0, upper=2*pi()> mu; // real<lower=0, upper=2*pi()> mu;
                real<lower=0, upper=1> rho;
            }}

            model {{
                mu ~ {self.mu.get_code()};
                rho ~ {self.rho.get_code()};

                real rho2 = square(rho);

                for (n in 1:N){{
                    target += -log(2*pi()) + log1m(rho2) - \
                        log1p(rho2 - 2*rho*cos(values[n] - mu));
                }}
            }}
        """
```

```

    return model_code

def get_parameters_prior(self) -> Dict[str, Any]:
    return {
        'mu': self.mu,
        'rho': self.rho
    }

```

F. Mixture class

```

from stan_circular_inference.factories.model_factory import ProbabilisticModel
from stan_circular_inference.factories.parameters_factory import MeanParameter, \
    VarianceParameter
from typing import Dict, Any

class Mixture(ProbabilisticModel):
    def __init__(self, dist1: ProbabilisticModel, dist2: ProbabilisticModel):
        super().__init__(f"Mixture_{dist1.name}_{dist2.name}")
        self.dist1 = dist1
        priors1 = dist1.get_parameters_prior()
        if self.dist1.name == 'Von Mises':
            self.dist1_mu = priors1['mu'].get_code()
            self.dist1_kappa = priors1['kappa'].get_code()
        else:
            self.dist1_mu = priors1['mu'].get_code()
            self.dist1_rho = priors1['rho'].get_code()
        self.dist2 = dist2
        priors2 = dist2.get_parameters_prior()
        if self.dist2.name == 'Von Mises':
            self.dist2_mu = priors2['mu'].get_code()
            self.dist2_kappa = priors2['kappa'].get_code()
        else:
            self.dist2_mu = priors2['mu'].get_code()
            self.dist2_rho = priors2['rho'].get_code()

    def gen_stan_model(self) -> str:
        model_code = f"""

        {'' if self.dist1.name == 'Von Mises' and self.dist2.name == 'Von Mises'
        else ''}
        functions{
            real cardioid_lpdf(real value, real mu, real rho) {
                return -log(2*pi()) + log1p(2*rho*cos(value - mu));
            }
        }
        """

```

```

    real wrapped_cauchy_lpdf(real value, real mu, real rho) {
      real rho2 = square(rho);
      return -log(2*pi()) + log1m(rho2) - log1p(rho2 - \
        2*rho*cos(value - mu));
    }
  }

  ''' if self.dist1.name == 'Cardioid' and self.dist2.name == 'Wrapped \
    Cauchy' self.dist1.name == 'Wrapped Cauchy' and \
    self.dist2.name == 'Cardioid'
  else '''
  functions{
    real cardioid_lpdf(real value, real mu, real rho) {
      return -log(2*pi()) + log1p(2*rho*cos(value - mu));
    }
  }

  ''' if self.dist1.name == 'Cardioid' or self.dist2.name == 'Cardioid'
  else '''
  functions{
    real wrapped_cauchy_lpdf(real value, real mu, real rho) {
      real rho_sqr = square(rho);
      return -log(2*pi()) + log1m(rho_sqr) - log1p(rho_sqr - \
        2*rho*cos(value - mu));
    }
  }

  '''}

  data {{
    int<lower=0> N;
    vector[N] values;
  }}

  parameters {{
    real<lower=0, upper=2*pi()> mu1;
    real<lower=0, upper=2*pi()> mu2;

    {'real<lower=0, upper=50> kappa1;' if self.dist1.name == 'Von Mises'
    else 'real<lower=0, upper=0.5> rho1;' if self.dist1.name == 'Cardioid'
    else 'real<lower=0, upper=1> rho1;'}
    {'real<lower=0, upper=50> kappa2;' if self.dist2.name == 'Von Mises'
    else 'real<lower=0, upper=0.5> rho2;' if self.dist2.name == 'Cardioid'
    else 'real<lower=0, upper=1> rho2;'}

    vector<lower=0, upper=1>[N] mixing_weight;
  }}

  model {{
    mixing_weight ~ beta(1, 1);
  }}

```

```

mu1 ~ {self.dist1_mu};
{f'kappa1 ~ {self.dist1_kappa}' if self.dist1.name == 'Von Mises' \
 else f'rho1 ~ {self.dist1_rho}'};

mu2 ~ {self.dist1_mu};
{f'kappa2 ~ {self.dist2_kappa}' if self.dist2.name == 'Von Mises' \
 else f'rho2 ~ {self.dist2_rho}'};

// Mixture model likelihood using log_sum_exp (it prevents \
underflow and overflow)
for (n in 1:N) {{
  target += log_sum_exp(
    log(mixing_weight[n]) + {'von_mises_lpdf(values[n] | mu1, \
kappa1)' if self.dist1.name == 'Von Mises'
                             else 'cardioid_lpdf(values[n] | \
mu1, rho1)' if \
                             self.dist1.name == 'Cardioid'
                             else 'wrapped_cauchy_lpdf( \
values[n] | mu1, rho1)'},
    log(1 - mixing_weight[n]) + {'von_mises_lpdf(values[n] | \
mu2, kappa2)' if self.dist2.name == 'Von Mises'
                                 else 'cardioid_lpdf(values[n] | \
mu2, rho2)' if \
                                 self.dist2.name == 'Cardioid'
                                 else 'wrapped_cauchy_lpdf( \
values[n] | mu2, rho2)'})
  });
}}
"""

return model_code

def get_parameters_prior(self) -> Dict[str, Any]:
  return {
    'mu': self.mu,
    'rho': self.rho
  }

```

Affiliation:

Diogo Almeida
Faculty of Sciences of the University of Lisbon
University at Lisbon
Campo Grande 016
1749-016 Lisboa, Portugal
E-mail: diogoalmeida107@hotmail.com

João Pedro Neto
LASIGE, DI, Faculty of Sciences of the University of Lisbon
University at Lisbon
Campo Grande 016
1749-016 Lisboa, Portugal
E-mail: jpNeto@ciencias.ulisboa.pt